



Universidad de Cuenca

Carrera de Ingeniería en Telecomunicaciones

Redes Inalámbricas

Sistema de Control de Acceso con RFID

Autores:

Elizabeth Corte
Eduardo Garcia
Jairo Galarza

Profesor:

Ing. Darwin Astudillo

Cuenca - Ecuador
9 de abril de 2026

Índice general

I.	Abstract	3
II.	Introducción	3
II.1.	Objetivos	3
III.	Marco Teórico	3
III.1.	Tecnología RFID y principio de funcionamiento	3
III.2.	Características principales de RFID	4
III.3.	Diferencias entre RFID y NFC	4
III.4.	Aplicaciones de RFID	4
III.5.	Seguridad en sistemas RFID	5
III.6.	Uso de servidores externos en sistemas RFID	5
IV.	Desarrollo	5
IV.1.	Diseño del sistema	5
IV.2.	Control de acceso con RFID	5
IV.3.	Conexión Wi-Fi al módulo del ESP32	6
IV.4.	Docker container	6
IV.5.	Servidor PHP	6
IV.6.	Circuito implementado	7
V.	Pruebas realizadas al sistema	8
VI.	Conclusiones	9
VII.	Bibliografía	10
VIII.	Anexos	11
VIII.1.	Código del SERVIDOR PHP	11
VIII.2.	Código implementado en ESP32	11

I. Abstract

This project presents the implementation of an access control system based on RFID technology. A reference system obtained from a microcontroller-based repository was adapted and replicated using Visual Studio Code with the assistance of GitHub Copilot to develop a server for data management. An ESP32 microcontroller was used to integrate the hardware components, including an RFID sensor, cards, and key fobs. When an RFID tag is detected, the system sends user identification and timestamp data to the server, enabling real-time access logging.

Keywords: RFID, ESP32, RC522, Server Communication, User Identification,

II. Introducción

En un contexto donde la seguridad y el control de acceso son cada vez más relevantes, el uso de tecnologías inteligentes se vuelve fundamental para garantizar sistemas eficientes y confiables. El presente trabajo describe el diseño e implementación de un sistema de control de acceso basado en tecnología RFID, orientado a la identificación de usuarios mediante tarjetas o llaveros electrónicos. A partir de un repositorio previamente desarrollado con otro microcontrolador, se realizó su adaptación empleando Visual Studio Code y el apoyo de GitHub Copilot para la generación de un servidor encargado de gestionar los registros de acceso.

El sistema utiliza un lector RFID RC522 para obtener el identificador único (UID) de cada dispositivo, el cual es enviado a una base de datos externa para validar si el acceso está autorizado. Dependiendo del resultado, se proporciona retroalimentación al usuario mediante un LED RGB y un buzzer, indicando el estado de la autenticación.

Además, cada intento de acceso es registrado junto con la fecha, hora, UID y estado de autorización, permitiendo un monitoreo eficiente en tiempo real. Para la implementación física, se empleó un ESP32 como unidad principal, integrando tanto la comunicación con el servidor como el control de los dispositivos periféricos. Este enfoque demuestra la viabilidad de desarrollar soluciones de control de acceso eficientes, escalables y de bajo costo.

II.1. Objetivos

II.1.1. Objetivo general

Implementar un sistema funcional de control de acceso utilizando la tecnología RFID junto con la pla-

taforma ESP32, integrando hardware, firmware y un servidor remoto de gestión de credenciales.

II.1.2. Objetivos específicos

- Describir el principio de funcionamiento de RFID y sus características (arquitectura, protocolo SPI, estándar ISO 14443).
- Diseñar e implementar el circuito de conexión RC522 + ESP32 + LED RGB + buzzer.
- Programar la lógica de control de acceso que verifique credenciales en una base de datos remota y registre cada evento.
- Evaluar la seguridad del sistema e identificar posibles vulnerabilidades con propuestas de mejora.

III. Marco Teórico

III.1. Tecnología RFID y principio de funcionamiento

La identificación por radiofrecuencia (RFID, *Radio Frequency Identification*) es una tecnología de identificación automática que permite reconocer objetos, personas o animales mediante ondas electromagnéticas, sin necesidad de contacto físico ni línea de vista directa. Un sistema RFID está compuesto principalmente por una etiqueta (tag), un lector y un sistema de procesamiento de datos. La etiqueta contiene un microchip y una antena, mientras que el lector emite señales de radiofrecuencia para activar y comunicarse con dicha etiqueta [?].

El funcionamiento del sistema se basa en la interacción electromagnética entre el lector y la etiqueta. Cuando el lector genera un campo electromagnético, la antena de la etiqueta lo capta y permite la activación del circuito interno del chip. Posteriormente, la etiqueta responde enviando la información almacenada mediante un proceso conocido como retrodispersión (*backscatter*), en el cual la señal es reflejada y modulada hacia el lector [?].



Figura 1: Funcionamiento básico de un sistema RFID.

En el caso de las etiquetas pasivas, que no disponen de batería, la energía necesaria para su funcionamiento se obtiene mediante transferencia de energía inductiva. Este fenómeno ocurre cuando el campo electromagnético generado por el lector induce una corriente en la antena de la etiqueta, permitiendo alimentar el circuito interno. Este mecanismo es fundamental en sistemas de corto alcance, especialmente en frecuencias como 13.56 MHz, donde el acoplamiento inductivo es predominante [?].

III.2. Características principales de RFID

La tecnología RFID presenta diversas características que determinan su desempeño y aplicaciones. Una de las más importantes es la frecuencia de operación, ya que influye directamente en el alcance, la velocidad de transmisión y la capacidad de penetración en distintos materiales. Las bandas más utilizadas son la baja frecuencia (LF), la alta frecuencia (HF) y la ultra alta frecuencia (UHF). Mientras que LF ofrece mayor resistencia a interferencias y se utiliza en aplicaciones de corto alcance, UHF permite mayores distancias de lectura y velocidades más altas, siendo ampliamente usada en logística y cadenas de suministro [?].

Otra característica relevante es el tipo de etiqueta. Las etiquetas pasivas son las más comunes debido a su bajo costo y mantenimiento reducido, aunque presentan menor alcance. Por otro lado, las etiquetas activas incorporan una batería interna, lo que les permite transmitir señales a mayores distancias, mientras que las semipasivas combinan características de ambas. Asimismo, el rango de lectura puede variar desde pocos centímetros hasta varios metros dependiendo de la frecuencia y del tipo de etiqueta utilizada [?].

En cuanto a estándares, existen diversas normativas internacionales que regulan la interoperabilidad de los sistemas RFID, como la familia ISO 14443 e ISO 15693 para alta frecuencia, y EPC Gen2 para aplicaciones en ultra alta frecuencia [?].

III.3. Diferencias entre RFID y NFC

Aunque NFC (*Near Field Communication*) es una evolución de RFID, existen diferencias importantes entre ambas tecnologías. RFID abarca múltiples frecuencias y protocolos, mientras que NFC opera exclusivamente en la banda de 13.56 MHz y utiliza estándares específicos como ISO 14443 e ISO 18092. Además, RFID generalmente se emplea en comunicaciones unidireccionales, donde el lector interroga a la etiqueta, mientras que NFC permite comunicación bidireccional entre dispositivos, lo que habilita aplicaciones más interactivas [?].



Figura 2: Comparación entre RFID y NFC

En términos de alcance, RFID puede operar a distancias de varios metros, especialmente en sistemas UHF, mientras que NFC está limitado a distancias muy cortas, típicamente menores a 10 cm. Esta limitación, aunque reduce su alcance, mejora la seguridad en aplicaciones como pagos electrónicos o autenticación. Por otro lado, RFID es más adecuado para la identificación masiva de objetos, mientras que NFC se orienta a aplicaciones de interacción directa con el usuario [?].

III.4. Aplicaciones de RFID

La tecnología RFID tiene una amplia variedad de aplicaciones en distintos sectores industriales y comerciales. En el ámbito logístico, se utiliza para la gestión de inventarios y el seguimiento de productos en tiempo real, lo que permite optimizar procesos y reducir errores humanos. En el sector salud, facilita la identificación de pacientes y el control de equipos médicos, mejorando la seguridad y eficiencia operativa.

Asimismo, RFID es fundamental en la trazabilidad de productos dentro de la cadena de suministro, permitiendo conocer el estado y ubicación de mercancías en todo momento. En el comercio minorista, se emplea para sistemas antirrobo y control de stock, mientras que en transporte se utiliza en sistemas de peaje electrónico y gestión de flotas [?].

III.5. Seguridad en sistemas RFID

A pesar de sus ventajas, los sistemas RFID presentan diversas vulnerabilidades de seguridad. Una de las principales es la clonación de etiquetas, especialmente en sistemas que utilizan identificadores únicos sin mecanismos de cifrado. También es posible la interceptación de la comunicación entre el lector y la etiqueta, así como ataques de repetición, en los que un atacante reutiliza datos capturados previamente para obtener acceso no autorizado [?].

Para mitigar estos riesgos, es fundamental implementar mecanismos de seguridad robustos. Una de las estrategias más efectivas es el uso de cifrado y autenticación mutua entre la etiqueta y el lector, lo que dificulta la clonación y el acceso indebido. Adicionalmente, se pueden emplear identificadores dinámicos o tokens temporales que cambien en cada transacción, reduciendo la posibilidad de ataques de repetición. Otra medida importante es trasladar la validación de acceso a un servidor externo, donde se pueden aplicar políticas de seguridad más avanzadas [?].

III.6. Uso de servidores externos en sistemas RFID

La utilización de una base de datos en un servidor externo, como Firebase, resulta fundamental en sistemas RFID modernos. Desde el punto de vista de la escalabilidad, un microcontrolador como el ESP32 presenta limitaciones en memoria y procesamiento, lo que restringe la cantidad de datos que puede manejar. En cambio, un servidor en la nube permite almacenar grandes volúmenes de información y gestionar múltiples usuarios simultáneamente.

En términos de disponibilidad, un servidor externo garantiza acceso continuo a los datos, incluso cuando el dispositivo local no está operativo. Esto es especialmente importante en aplicaciones de monitoreo en tiempo real. Además, desde la perspectiva de seguridad, los servidores permiten implementar mecanismos avanzados como autenticación, cifrado de datos y control de acceso, lo que no es viable en sistemas embebidos de recursos limitados [?].

IV. Desarrollo

IV.1. Diseño del sistema

El sistema desarrollado consiste en un control de acceso basado en tecnología RFID, implementado con un microcontrolador ESP32 y un lector MFRC522. Su funcionamiento se basa en la lectura del identificador único (UID) de cada tarjeta o etiqueta RFID, el cual

es enviado mediante una solicitud HTTP a un servidor local alojado en un entorno XAMPP.

En el servidor, una aplicación desarrollada en PHP consulta una base de datos MySQL para verificar si el UID recibido corresponde a una tarjeta autorizada. En función del resultado de esta validación, el servidor devuelve una respuesta al ESP32, indicando si el acceso debe ser concedido o denegado.

Cuando el acceso es autorizado, el ESP32 activa un servomotor que simula o acciona el mecanismo de apertura, además de señalar el evento mediante un LED y un buzzer. En caso contrario, el sistema indica acceso denegado mediante señales visuales y sonoras. De forma complementaria, el sistema puede registrar cada intento de acceso para fines de supervisión y control.

Este esquema está basado en una arquitectura IoT de control de acceso, similar a la empleada en proyectos que integran ESP32, RC522 y una interfaz web con PHP/MySQL para la administración de credenciales y el monitoreo de eventos.

IV.2. Control de acceso con RFID

El módulo RFID MFRC522 requiere inicializar la comunicación SPI, un protocolo de comunicación serial utilizado para el intercambio de datos entre dispositivos en un bus SPI. Este paso configura el sistema para permitir la comunicación con el lector de tarjetas RFID y constituye la etapa inicial del programa.

Posteriormente, se verifican las características del lector con el fin de detectar posibles anomalías que puedan afectar su funcionamiento.

Una vez identificado el NUID de la tarjeta, se procede al control de acceso. Para ello, se compara el identificador con una base de datos de usuarios autorizados. Cada usuario está asociado a un nombre, y mediante condiciones lógicas se determina si la tarjeta corresponde a un acceso permitido, así como si se trata de una acción de ingreso o salida.

Estas verificaciones se realizan dentro de un bucle, en el cual también se registra la hora en que ocurre el evento. La obtención de la fecha y hora se realiza mediante un servidor NTP, cuyo funcionamiento se detalla en la siguiente sección.

Adicionalmente, el sistema detecta lecturas repetidas de una misma tarjeta. Para ello, se almacena temporalmente el NUID leído y se compara con el siguiente. Si ambos coinciden, se genera un mensaje indicando que la tarjeta ya ha sido registrada previamente.

En resumen, la lectura y procesamiento del NUID constituye la base del sistema de control de acceso implementado.



Figura 3: Módulo RC522

IV.3. Conexión Wi-Fi al módulo del ESP32

Para habilitar la conectividad del sistema, se utiliza la librería `WiFi.h`, la cual permite al módulo ESP32 conectarse a una red inalámbrica. Esta conexión es necesaria para establecer comunicación con el servidor local, encargado de procesar la información enviada por el sistema de control de acceso.

Una vez conectado a la red Wi-Fi, el ESP32 utiliza la librería `HTTPClient.h` para realizar solicitudes HTTP al servidor. En cada lectura válida de una tarjeta RFID, el identificador único (UID) es capturado y enviado mediante una petición al servidor a través de una URL previamente definida.

El servidor, que se encuentra desplegado localmente, recibe el UID y determina si el acceso debe ser concedido o denegado. En función de la respuesta recibida, el ESP32 activa los indicadores correspondientes, como los LED, el buzzer y el servomotor, permitiendo la comunicación entre el lector RFID y el servidor de control de acceso.

IV.4. Docker container

El sistema de control de acceso implementado utiliza un servidor local desplegado mediante contenedores Docker. Esta tecnología permite encapsular la aplicación del servidor junto con sus dependencias en un entorno aislado, facilitando su ejecución, mantenimiento y portabilidad. En la Fig.4 se evidencia que nuestro servicio esta levantado.

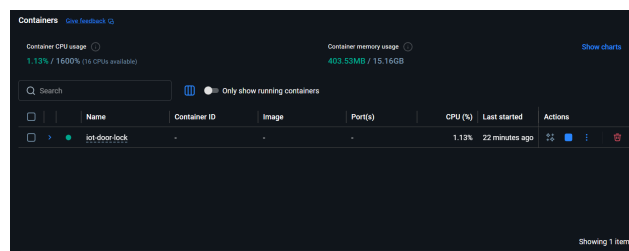


Figura 4: Docker container

En este proyecto, el servidor se encarga de recibir las solicitudes enviadas por el módulo ESP32 a través de la red Wi-Fi. Cada vez que se detecta una tarjeta RFID, el identificador único (UID) es transmitido mediante una petición HTTP hacia el servidor, el cual procesa dicha información para determinar si el acceso debe ser concedido o denegado.

El contenedor Docker, se encuentra en ejecución, lo que garantiza la disponibilidad continua del servicio. Este servidor actúa como intermediario entre el hardware (ESP32 + RFID) y la lógica de control de acceso, permitiendo centralizar la gestión de usuarios y registros.

Una de las principales ventajas del uso de Docker es la capacidad de desplegar el sistema de forma rápida y consistente en distintos entornos, sin necesidad de configuraciones complejas. Además, el aislamiento proporcionado por los contenedores mejora la seguridad y estabilidad del sistema.

Por medio de esta arquitectura, el sistema no depende de servicios externos, ya que todo el procesamiento se realiza de manera local, lo que reduce la latencia y mejora la confiabilidad del control de acceso.

IV.5. Servidor PHP

El sistema de control de acceso implementado utiliza un servidor web local basado en PHP, el cual se ejecuta en el entorno `localhost`. Este servidor se encuentra desplegado dentro de un contenedor Docker, lo que permite encapsular tanto el servicio web como la base de datos en un entorno controlado y portátil.

El archivo `admin.php` constituye la interfaz principal del sistema, desde donde se gestionan las credenciales RFID, se visualiza el historial de accesos y se ejecutan acciones de control manual, como la apertura de la puerta.

El servidor establece una conexión con una base de datos MySQL mediante la extensión `mysqli`, utilizando parámetros como el host, usuario, contraseña y nombre de la base de datos. En este caso, el host se define como `localhost`, lo que indica que la base de datos se encuentra en el mismo entorno del servidor, facilitando una comunicación rápida y eficiente.

Una de las funcionalidades principales del sistema es la gestión de credenciales. A través de formularios web, es posible registrar nuevos identificadores RFID (UID) asociados a un usuario, así como eliminar registros existentes. Esta información se almacena en la tabla **users** de la base de datos.

Adicionalmente, el sistema mantiene un historial de accesos, almacenado en la tabla **logs**, donde se registra cada intento de acceso junto con su estado (permitido o denegado) y la marca temporal correspondiente. Estos datos pueden ser exportados en formato CSV para su posterior análisis.

El sistema también implementa un mecanismo de autenticación de administrador mediante sesiones `sessionstart`, lo que permite restringir el acceso a la interfaz administrativa. Asimismo, se incluye la opción de cierre de sesión para garantizar la seguridad del sistema.

Otra característica importante es el manejo de zonas horarias. Aunque los registros en la base de datos se almacenan en formato UTC, el sistema convierte estos valores a la zona horaria local (America/Guayaquil) antes de mostrarlos, asegurando una correcta interpretación de los eventos.

Finalmente, el servidor permite el control manual de la puerta mediante la generación de un archivo de comando (`door_command.txt`), el cual puede ser interpretado por otro proceso del sistema para ejecutar la acción correspondiente.

En conjunto, el servidor PHP en `localhost` actúa como el núcleo del sistema, encargándose de la gestión de datos, la lógica de control y la interacción con el usuario a través de una interfaz web.

En la Fig.5 se puede verificar el servidor ya en línea.

Figura 5: Inicio de sesión del administrador

Dentro del servicio tenemos el siguiente panel, donde podemos agregar a los usuarios y además se almacenan los registros de acuerdo si se concedió o no el acceso, esto se puede verificar en la Fig.6

Gestion de credenciales

UID de la credencial: Nombre del usuario: [Agregar credencial](#)

Credenciales registradas

ID	UID	Nombre	Fecha	Acciones
1	43249CC5	LLAVERO	2026-04-03 12:12:38	Eliminar

Historial de accesos

[Exportar CSV](#)

ID	UID	Estado	Fecha/Hora
89	43249CC5	Acceso permitido	2026-04-03 14:21:08
88	13C652F6	Acceso denegado	2026-04-03 14:21:04
87	43249CC5	Acceso permitido	2026-04-03 14:20:58

Figura 6: Interfaz del servidor

IV.6. Circuito implementado

El sistema implementado hace uso de diversos pines del microcontrolador ESP32 para la conexión de periféricos como el lector RFID, indicadores visuales, actuadores y elementos de señalización. A continuación, se detalla la función de cada uno de los pines utilizados:

Pines del módulo RFID (MFRC522):

- **SS_PIN (GPIO 5):** Corresponde al pin SDA del módulo RFID, utilizado como señal de selección de esclavo (Slave Select) en la comunicación SPI.
- **RST_PIN (GPIO 22):** Pin de reinicio del módulo RFID, permite reiniciar el funcionamiento del lector cuando sea necesario.
- **SCK_PIN (GPIO 18):** Señal de reloj del bus SPI (Serial Clock), sincroniza la transmisión de datos entre el ESP32 y el módulo RFID.
- **MISO_PIN (GPIO 19):** Línea de datos desde el esclavo hacia el maestro (Master In Slave Out), utilizada para que el RFID envíe datos al ESP32.
- **MOSI_PIN (GPIO 23):** Línea de datos desde el maestro hacia el esclavo (Master Out Slave In), utilizada para enviar datos desde el ESP32 al módulo RFID.

Pines de salida e indicadores:

- **READY_LED_PIN (GPIO 25):** Indica que el sistema está encendido y listo para operar.
- **GRANTED_LED_PIN (GPIO 26):** Se activa cuando el acceso es concedido.
- **DENIED_LED_PIN (GPIO 27):** Se activa cuando el acceso es denegado.
- **BUZZER_PIN (GPIO 33):** Controla un buzzer que emite señales acústicas para indicar el resultado del acceso.

Pin de control del actuador:

- **SERVO_PIN (GPIO 32):** Se utiliza para controlar un servomotor (SG90) mediante una señal PWM. Este actuador permite simular la apertura y cierre de una puerta en función del resultado del acceso.

El servomotor es controlado mediante el periférico PWM del ESP32, configurado a una frecuencia de 50 Hz y una resolución de 16 bits, lo cual permite generar señales precisas para posicionar el eje del servo en diferentes ángulos.

En conjunto, la correcta asignación y uso de estos pines permite la integración eficiente de los diferentes componentes del sistema, garantizando una comunicación adecuada con el módulo RFID y una respuesta inmediata mediante actuadores e indicadores. El circuito implementado se puede verificar en la Fig.7

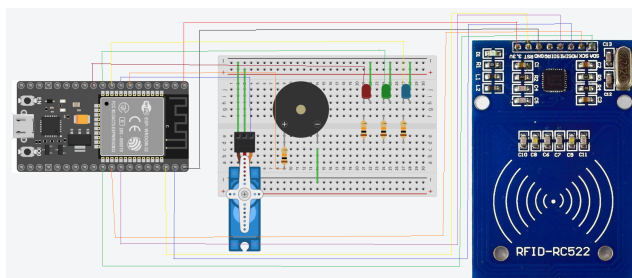


Figura 7: Circuito implementado

V. Pruebas realizadas al sistema

1. Registre al menos dos UUIDs en la base de datos: uno autorizado y uno no autorizado.

Se registro diferentes UUIDs, los cuales se pueden verificar en la siguiente Fig.8, además se tienen UUIDs no registrados los cuales no se pueden evidenciar en esta Figura.

Gestion de credenciales

Credenciales registradas

ID	UID	Nombre	Fecha	Acciones
7	05AFB90E	Carnet estudiantil	2026-04-08 20:18:00	Eliminar
4	F9F508B0	BUS TARJETA	2026-04-06 21:41:23	Eliminar
1	43249CC5	LLAVERO	2026-04-03 12:12:38	Eliminar

Figura 8: UUIDs registrados

2. Pruebe ambos casos y verifique la respuesta visual (LED) y sonora (buzzer).

En la Fig.9 se puede evidenciar como se identificó una UID que no está registrado en el cual se puede verificar el led de color rojo.

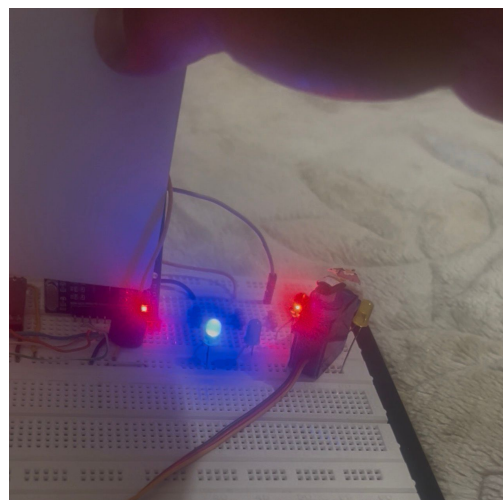


Figura 9: Sistema denegado

Ahora podemos verificar el led de color verde en la Fig.10, significa que el sistema fue concedido, es decir esta UID está registrado en el servidor.

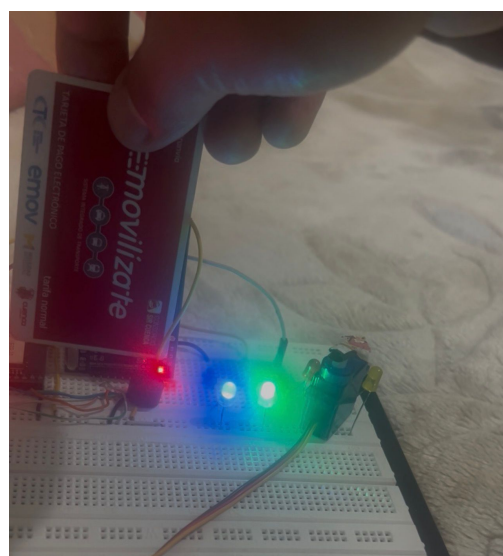


Figura 10: Sistema concedido

3. Consulte la ruta `/log` y confirme que los eventos quedan registrados con fecha, hora y UID.

Se configuro el uso horario del servidor de acuerdo al siguiente código en la implementación de este, se puede verificar en las siguientes líneas:

Ahora en la Fig.11 se puede evidenciar como el sistema registra fechas y horas de acuerdo al uso horario y nuestra locación.

Fecha/Hora
2026-04-08 20:19:01
2026-04-08 20:18:52
2026-04-08 20:18:28
2026-04-08 20:18:19
2026-04-08 20:18:05
2026-04-08 20:17:42
2026-04-08 20:17:10

Figura 11: Fecha/Hora

4. Prueba de fallo de red: desconecte el Wi-Fi o detenga el servidor y documente el comportamiento del sistema.

Se realizo las posibles fallas al sistema, de cualquier manera, ya sea perdiendo la conexión Wi-Fi o apagando el servidor, tenemos una falla en el sistema, este ya no detecta ninguna UIDs, debido a que el sistema por medio del ESP32 está conectado a la misma red, mientras no exista conexión a la red este no funcionará.

5. Prueba de escalabilidad: agregue cinco usuarios y realice diez lecturas consecutivas midiendo el tiempo de respuesta. Se realizo 10 pruebas consecutivas y la reacción del sistema fue inmediata, no se tuvo problemas con la respuesta del sistema, reacciono al mismo tiempo en cada una de las diferentes iteraciones, a continuación, en la Fig.12 se puede evidenciar las 10 iteraciones seguidas.

115	13C652F6	Acceso denegado
114	13C652F6	Acceso permitido
113	43249CC5	Acceso permitido
112	13C652F6	Acceso permitido
111	13C652F6	Acceso permitido
110	13C652F6	Acceso permitido
109	13C652F6	Acceso permitido
108	43249CC5	Acceso permitido
107	13C652F6	Acceso permitido
106	13C652F6	Acceso permitido
105	13C652F6	Acceso permitido

Figura 12: Iteraciones seguidas

Finalmente tenemos el siguiente enlace al repositorio de GitHub, donde tenemos archivos importantes empleados en este proyecto: <https://github.com/wilsonnegarcia-cmd/IOT-door-lock--Miniproyecto-1->

VI. Conclusiones

- Se logró implementar un sistema funcional que integra componentes de hardware (microcontrolador ESP32, módulo RFID RC522 y servomotor) con una arquitectura de software backend. Esto demuestra la viabilidad de conectar dispositivos físicos embebidos con servicios web en tiempo real.
- La decisión de utilizar un servidor PHP (alojado en un contenedor Docker) para la validación de los UID, en lugar de almacenar las credenciales de forma local (hardcoded) en el ESP32, proporciona un sistema altamente escalable y seguro. Esto permite actualizar la base de datos de usuarios autorizados de forma remota sin necesidad de reprogramar el microcontrolador.
- Dado que el sistema utiliza el protocolo HTTP sobre una conexión Wi-Fi para consultar la autorización (GET request), se concluye que el tiempo de respuesta y la fiabilidad del control de acceso (apertura del servo) dependen directamente de la latencia y estabilidad de la red inalámbrica local. Se comprobó la necesidad de rutinas de manejo de errores (timeouts) para evitar bloqueos en el sistema si el servidor no responde.

- La práctica demuestra que es posible desarrollar un sistema robusto de control de acceso físico e identificación utilizando tecnologías asequibles, manteniendo un nivel de seguridad adecuado para aplicaciones de domótica, entornos universitarios o empresariales de bajo riesgo.

VII. Bibliografía

- [1] K. Finkenzeller, *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards and Identification*, 3rd ed., Wiley, 2010.
- [2] D. M. Dobkin, *The RF in RFID: Passive UHF RFID in Practice*, Elsevier, 2012.
- [3] C. A. Balanis, *Antenna Theory: Analysis and Design*, 4th ed., Wiley, 2016.
- [4] K. Klaus, "Security in RFID and NFC Systems," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 2, pp. 1010–1023, 2014.

VIII. Anexos

VIII.1. Código del SERVIDOR PHP

```
$app_timezone_name = getenv('APP_TIMEZONE') ?:
    'America/Guayaquil';
date_default_timezone_set($app_timezone_name);

$utc_timezone = new DateTimeZone('UTC');
$app_timezone = new DateTimeZone($app_timezone_name);

function format_utc_timestamp(string $timestamp,
    DateTimeZone $utc_timezone, DateTimeZone
    $app_timezone): string
{
    $date_time = DateTime::createFromFormat('Y-m-d
    H:i:s', $timestamp, $utc_timezone);

    if ($date_time === false) {
        return $timestamp;
    }

    $date_time->setTimezone($app_timezone);
    return $date_time->format('Y-m-d H:i:s');
}

$db_host = getenv('DB_HOST') ?: 'localhost';
$db_user = getenv('DB_USER') ?: 'root';
$db_pass = getenv('DB_PASS') ?: '';
$db_name = getenv('DB_NAME') ?: 'rfid_access';
$db = new mysqli($db_host, $db_user, $db_pass,
    $db_name);

if ($db->connect_error) {
    die("Error de conexion MySQL: " .
        $db->connect_error);
}

$db->query("SET time_zone = '+00:00'");

// Exportacion CSV
if (isset($_GET['export'])) {
    header('Content-Type: text/csv');
    header('Content-Disposition: attachment;
        filename="logs_acces_'.date('Y-m-d').'.csv"');

    $output = fopen('php://output', 'w');
    fputcsv($output, ['ID', 'UID', 'Estado', 'Fecha y
        hora'], ';');

    $logs = $db->query("SELECT * FROM logs ORDER BY
        timestamp DESC");
    while ($row = $logs->fetch_assoc()) {
        fputcsv($output, [
            $row['id'],
            $row['uid'],
            $row['access_status'],
            format_utc_timestamp($row['timestamp'],
                $utc_timezone, $app_timezone)
        ], ';');
    }
    fclose($output);
    exit;
}

// Login
if (isset($_POST['login'])) {
```

```
if ($_POST['username'] === 'admin' &&
    $_POST['password'] === 'admin123') {
    $_SESSION['loggedin'] = true;
} else {
    $error = "Credenciales incorrectas";
}

// Logout
if (isset($_GET['logout'])) {
    session_destroy();
    header("Location: admin.php");
    exit;
}

// Agregar credencial
if (isset($_POST['add_badge'])) {
    $uid = $db->real_escape_string($_POST['new_uid']);
    $name =
        $db->real_escape_string($_POST['user_name']);
    $db->query("INSERT INTO users (uid, name) VALUES
        ('$uid', '$name')");
}

// Eliminar credencial
if (isset($_POST['delete_badge'])) {
    $id = intval($_POST['delete_id']);
    $db->query("DELETE FROM users WHERE id = $id");
}

// Abrir puerta
if (isset($_POST['open_door'])) {
    file_put_contents('door_command.txt', 'OPEN');
}
?>
```

Listing 1: Interfaz administrativa en PHP para el control de acceso RFID.

VIII.2. Código implementado en ESP32

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <SPI.h>
#include <MFRC522.h>

// Pines RFID (adaptados para ESP32)
#define SS_PIN 5 // SDA
#define RST_PIN 22 // RST
#define SCK_PIN 18 // SPI clock
#define MISO_PIN 19 // SPI MISO
#define MOSI_PIN 23 // SPI MOSI
#define READY_LED_PIN 25 // Sistema listo
#define GRANTED_LED_PIN 26 // Acceso concedido
#define DENIED_LED_PIN 27 // Acceso denegado
#define SERVO_PIN 32 // Senal del servomotor
SG90
#define BUZZER_PIN 33 // Buzzer
#define SERVO_PWM_CHANNEL 0
#define SERVO_PWM_FREQUENCY 50
#define SERVO_PWM_RESOLUTION 16

// Datos WiFi (AP compartido)
const char* ssid = "ANGEL_SARANGO";
const char* password = "Suquinancy77";
```

```
// Host del servidor local (evita depender de una IP fija)
// Si el laboratorio no resuelve nombres de equipo, usa una reserva DHCP o vuelve a IP fija.
const char* serverHost = "192.168.0.104";
const uint16_t serverPort = 80;
const char* serverPath = "/api.php";
const unsigned long timeout = 5000; // Tiempo de espera HTTP en ms
const unsigned int accessLedDurationMs = 1500;
const unsigned int grantedBuzzerDurationMs = 200;
const unsigned int deniedBuzzerDurationMs = grantedBuzzerDurationMs * 3;
const unsigned int servoOpenDurationMs = 1000;
const int servoRestAngle = 0;
const int servoOpenAngle = 90;
const unsigned long wifiConnectTimeoutMs = 20000;

MFR522 rfid(SS_PIN, RST_PIN); // Inicializacion del lector RFID

const char* wifiStatusToString(wl_status_t status) {
    switch (status) {
        case WL_IDLE_STATUS:
            return "IDLE_STATUS";
        case WL_NO_SSID_AVAIL:
            return "NO_SSID_AVAIL";
        case WL_SCAN_COMPLETED:
            return "SCAN_COMPLETED";
        case WL_CONNECTED:
            return "CONNECTED";
        case WL_CONNECT_FAILED:
            return "CONNECT_FAILED";
        case WL_CONNECTION_LOST:
            return "CONNECTION_LOST";
        case WL_DISCONNECTED:
            return "DISCONNECTED";
        default:
            return "UNKNOWN";
    }
}

String buildServerUrl(const String& uid) {
    return "http://" + String(serverHost) + ":" + String(serverPort) + String(serverPath) + "?uid=" + uid;
}

void logRfidStatus() {
    byte version = rfid.PCD_ReadRegister(MFR522::VersionReg);

    Serial.printf("RFID pins -> SS:%d RST:%d SCK:%d MISO:%d MOSI:%d\n", SS_PIN, RST_PIN, SCK_PIN, MISO_PIN, MOSI_PIN);

    if (version == 0x00 || version == 0xFF) {
        Serial.println("RC522 no detectado. Revisa cableado, 3.3V, GND y pines SPI");
    } else {
        Serial.printf("RC522 detectado. Version: 0x%02X\n", version);
    }
}

void logServerResolution() {
    IPAddress serverIp;
```

```
if (WiFi.hostByName(serverHost, serverIp)) {
    Serial.println("Servidor resuelto: " + String(serverHost) + " -> " + serverIp.toString());
} else {
    Serial.println("No se pudo resolver el host del servidor: " + String(serverHost));
    Serial.println("La red actual probablemente no publica DNS local para ese nombre");
}

bool isTargetNetworkVisible() {
    int networkCount = WiFi.scanNetworks();

    if (networkCount <= 0) {
        Serial.println("No se detectaron redes WiFi durante el escaneo");
        return false;
    }

    bool networkFound = false;

    Serial.println("Redes detectadas:");
    for (int i = 0; i < networkCount; i++) {
        String currentSsid = WiFi.SSID(i);
        Serial.printf("  %d) %s RSSI: %d dBm Canal: %d\n", i + 1, currentSsid.c_str(), WiFi.RSSI(i), WiFi.channel(i), (WiFi.encryptionType(i) == WIFI_AUTH_OPEN) ? "abierta" : "protegida");

        if (currentSsid.equals(ssid)) {
            networkFound = true;
        }
    }

    return networkFound;
}

bool connectToWifi() {
    WiFi.mode(WIFI_STA);
    WiFi.setSleep(false);
    WiFi.disconnect(true, true);
    delay(200);

    bool networkVisible = isTargetNetworkVisible();
    if (!networkVisible) {
        Serial.println("La red configurada no aparece en el escaneo: " + String(ssid));
        Serial.println("Verifica que el hotspot/router este emitiendo en 2.4 GHz y que el SSID sea exacto");
    }

    if (password[0] == '\0') {
        WiFi.begin(ssid);
    } else {
        WiFi.begin(ssid, password);
    }
    Serial.print("Conectando a WiFi");

    unsigned long startTime = millis();
    wl_status_t lastStatus = WL_IDLE_STATUS;

    while (millis() - startTime < wifiConnectTimeoutMs) {
        wl_status_t currentStatus = WiFi.status();
```

```

    if (currentStatus == WL_CONNECTED) {
        Serial.println();
        return true;
    }

    if (currentStatus != lastStatus) {
        Serial.printf("\nEstado WiFi: %s\n",
            wifiStatusToString(currentStatus));
        lastStatus = currentStatus;
    }

    delay(500);
    Serial.print(".");
}

Serial.println();
Serial.printf("Fallo WiFi. Estado final: %s\n",
    wifiStatusToString(WiFi.status()));
return false;
}

uint32_t angleToDutyCycle(int angle) {
    const uint32_t minPulseUs = 500;
    const uint32_t maxPulseUs = 2400;
    const uint32_t pwmMaxDuty = (1UL <<
        SERVO_PWM_RESOLUTION) - 1;
    const uint32_t pulseUs = minPulseUs + ((maxPulseUs -
        minPulseUs) * (uint32_t)angle) / 180U;
    const uint32_t periodUs = 1000000UL /
        SERVO_PWM_FREQUENCY;

    return (pulseUs * pwmMaxDuty) / periodUs;
}

void setServoAngle(int angle) {
    if (angle < 0) {
        angle = 0;
    } else if (angle > 180) {
        angle = 180;
    }

    ledcWrite(SERVO_PWM_CHANNEL,
        angleToDutyCycle(angle));
}

void initializeServo() {
    ledcSetup(SERVO_PWM_CHANNEL, SERVO_PWM_FREQUENCY,
        SERVO_PWM_RESOLUTION);
    ledcAttachPin(SERVO_PIN, SERVO_PWM_CHANNEL);
    setServoAngle(servoRestAngle);
}

void activateServoLock() {
    Serial.println("Servo ACTIVADO");
    setServoAngle(servoOpenAngle);
    delay(servoOpenDurationMs);
    setServoAngle(servoRestAngle);
    Serial.println("Servo EN REPOSO");
}

void initializeOutputs() {
    pinMode(READY_LED_PIN, OUTPUT);
    pinMode(GRANTED_LED_PIN, OUTPUT);
    pinMode(DENIED_LED_PIN, OUTPUT);
    pinMode(BUZZER_PIN, OUTPUT);

    digitalWrite(READY_LED_PIN, HIGH);
    digitalWrite(GRANTED_LED_PIN, LOW);
    digitalWrite(DENIED_LED_PIN, LOW);

    digitalWrite(BUZZER_PIN, LOW);
}

void signalAccessResult(int ledPin, unsigned int
    buzzerDurationMs) {
    digitalWrite(ledPin, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);
    delay(buzzerDurationMs);
    digitalWrite(BUZZER_PIN, LOW);
    delay(accessLedDurationMs);
    digitalWrite(ledPin, LOW);
}

void signalAccessGranted() {
    signalAccessResult(GRANTED_LED_PIN,
        grantedBuzzerDurationMs);
}

void signalAccessDenied() {
    signalAccessResult(DENIED_LED_PIN,
        deniedBuzzerDurationMs);
}

void setup() {
    Serial.begin(115200);
    delay(1000);

    SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, SS_PIN); //
        Inicializacion SPI explicita para ESP32
    delay(50);
    rfid.PCD_Init(); // Inicializacion del modulo
        RFID
    logRfidStatus();

    initializeOutputs();
    initializeServo();

    if (!connectToWifi()) {
        Serial.println("Error de conexion WiFi");
    } else {
        Serial.println("Conectado. IP: " +
            WiFi.localIP().toString());
        logServerResolution();
    }
}

void loop() {
    // Verifica si hay una nueva tarjeta
    if (!rfid.PICC_IsNewCardPresent() ||
        !rfid.PICC_ReadCardSerial()) {
        delay(10);
        return;
    }

    // Leer el UID de la tarjeta
    String uid = "";
    for (byte i = 0; i < rfid.uid.size; i++) {
        char buffer[3];
        sprintf(buffer, "%02X", rfid.uid.uidByte[i]);
        uid += buffer;
    }

    Serial.println("UID detectado: " + uid);

    // Enviar el UID al servidor
    if (WiFi.status() == WL_CONNECTED) {
        WiFiClient client;
        HTTPClient http;

```

```
// Construir la URL con el parametro GET
String fullUrl = buildServerUrl(uid);
Serial.println("Consultando URL: " + fullUrl);
http.begin(client, fullUrl);
http.setTimeout(timeout);

int httpCode = http.GET();

if (httpCode == HTTP_CODE_OK) {
    String response = http.getString();
    response.trim();
    Serial.println("Respuesta del servidor: " +
response);

    if (response.equalsIgnoreCase("GRANTED")) {
        signalAccessGranted();
        activateServoLock();
    } else {
        Serial.println("Acceso denegado");
        signalAccessDenied();
    }
} else {
    Serial.printf("Error HTTP: %d (%s)\n", httpCode,
http.errorToString(httpCode).c_str());
}

http.end();
} else {
    Serial.println("Sin conexion WiFi");
}

// Fin de la comunicacion con la tarjeta
rfid.PICC_HaltA();
rfid.PCD_StopCrypto1();
delay(200);
}
```

Listing 2: Código del ESP32 para el sistema de control de acceso RFID.